GLUCOTWIN

Architecture Workflow & System Diagram

Multimodal glucose-risk prediction for Type 2 Diabetes

PURPOSE
This document explains the GlucoTwin prototype architecture, end-to-end data workflow, major technical components, and evaluation and safety considerations. It is technical project documentation, not evidence of clinical validation.

PROJECT DOCUMENTATION • PROTOTYPE ARCHITECTURE • 20 OCTOBER 2026

1. System Architecture Diagram

The diagram shows the main sequence from data acquisition to application delivery. Evaluation and platform services support the end-to-end workflow.

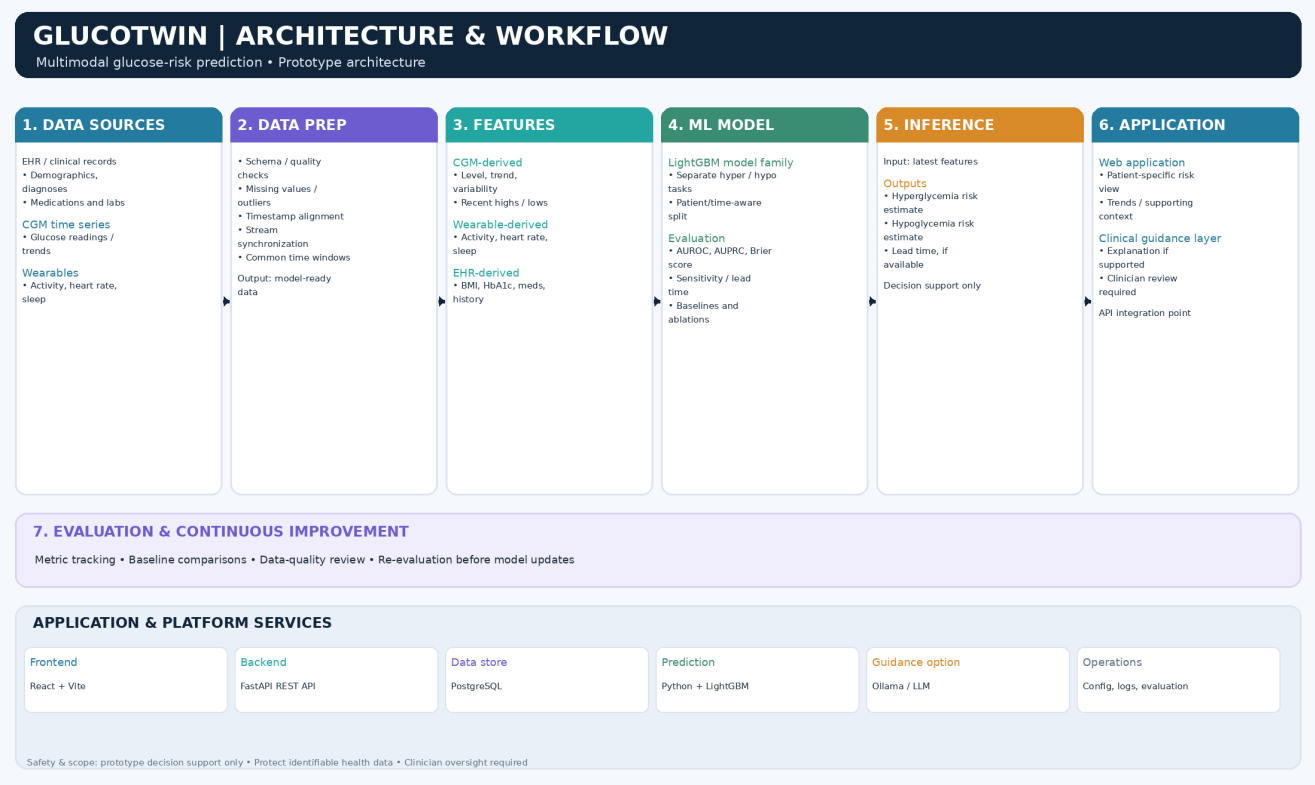


Figure 1. High-level architecture. Confirm exact data fields, prediction horizons, and deployment boundaries against the current implementation.

2. End-to-End Architecture Workflow

Step 1 — Data sources

The workflow can combine electronic health record (EHR) information, continuous glucose monitoring (CGM) time series, and wearable signals when available. EHR data may include demographics, diagnoses, medications, laboratory results, and history. CGM data provides glucose measurements and temporal trends. Wearables may provide activity, heart-rate, and sleep-related signals.

Step 2 — Data preparation

Inputs are checked for schema consistency and quality. The preparation stage handles or flags missing values and outliers, aligns timestamps, synchronizes available streams, and creates consistent time windows. Quality issues should be flagged rather than silently treated as valid measurements.

Step 3 — Feature engineering

Features may include current glucose level, trend and variability; activity, heart-rate and sleep summaries; and selected clinical context such as BMI, HbA1c, medications, comorbidities, and prior history. Only fields actually present and validated in the implementation should be described as active inputs.

Step 4 — Model training

The documented prototype uses a LightGBM model family for separate hyperglycemia and hypoglycemia prediction tasks. Training and evaluation should avoid patient leakage and respect time order where applicable. Target labels and the prediction horizon must be explicitly defined.

Step 5 — Model evaluation

Use complementary metrics such as AUROC, AUPRC, Brier score, sensitivity at a defined operating point, and lead-time analysis where supported. Baselines and feature ablations can test the contribution of additional data sources. Metrics must come from a verified, consistent evaluation run.

Step 6 — Inference

At inference time, the latest validated features are passed to the model service. Outputs may include hyperglycemia and hypoglycemia risk estimates and an estimated lead time if supported. Present outputs with relevant context, data-quality caveats, and model-version information.

Step 7 — Application and review

The frontend presents patient-specific outputs and context. A backend API coordinates requests, data access, and inference. Any language-model guidance layer is optional and must be grounded in available evidence. Clinical interpretation and decisions remain with qualified professionals.

Step 8 — Monitoring and improvement

Track data quality, model performance, input-distribution changes, and versioned evaluation results. Evaluate updates before release. Reviewer feedback can inform improvements but should not automatically be treated as ground truth.

3. Main Technical Components

Component	Technology	Role
Frontend	React + Vite	Web interface and patient-specific views.
Backend API	FastAPI	REST endpoints and orchestration between UI, data store, and inference.
Data store	PostgreSQL	Structured storage for application and patient/event records, according to configured schema and access controls.
ML inference	Python + LightGBM	Model training/evaluation and risk-estimation workflow.
Optional guidance	Ollama / LLM layer	Optional contextual language output if configured; must not invent clinical facts or replace clinician review.
Operations	Evaluation scripts, metrics, logs, versioning	Repeatable evaluation, monitoring, debugging, and traceability.

4. Data Flow and Interfaces

The frontend sends requests to the backend API. The backend validates requests, accesses authorized records, prepares model inputs, invokes prediction logic, and returns estimates and context to the frontend. Optional language-model guidance should receive only minimum necessary context and operate within explicit safety and privacy constraints.

5. Evaluation and Reporting

A defensible report states data source, patient/time split, target labels, prediction horizon, sample counts, class balance, model version, baseline results, and metric definitions. For imbalanced outcomes, consider AUPRC and calibration alongside AUROC. Lead-time results require a clear definition and unit. Do not combine headline metrics from different evaluation runs without explanation; development or synthetic evaluation is not proof of clinical performance.

6. Safety, Privacy, and Limitations

Prototype status: no regulatory clearance or clinical validation is implied. Human oversight is required. Missing, delayed, inaccurate, or misaligned data and population/device shifts may affect estimates. Protect identifiable health information. Generated explanations must be grounded in available records and model outputs.

7. Summary

GlucoTwin follows a modular workflow: acquire available patient signals, validate and synchronize data, derive features, train and evaluate prediction models, serve risk estimates, and present outputs for human review. Credibility depends on reproducible evaluation, transparent limitations, privacy safeguards, and qualified clinical oversight. Verify every listed input, service, and deployment control against the current repository before submission.